

|      |                         |
|------|-------------------------|
| 任务号  | 5                       |
| 姓名   | 杨力泉 元杰                  |
| 学号   | 231220069 231220082     |
| 日期   | 2026 年 5 月 11 日         |
| 附加任务 | 选做4.2：一维数组作为函数参数 + 高维数组 |

# 实验报告 — 中间代码生成器实现与调试

## 一、实验目标与概述

本实验目标是实现一个 C-- 语言到三地址码形式中间代码的翻译器。在前两个实验（词法/语法分析、语义分析）的基础上，对已有 AST 进行语法制导翻译，输出符合教材表 4.6 规范的中间代码。必做部分要求覆盖基础数据类型、算术/关系/逻辑表达式、赋值语句、一维数组、控制流语句、函数定义与调用；选做 4.2 放宽限制，额外支持高维数组的声明与访问以及一维数组作为函数参数（传地址）。

本次实现新增/修改的核心文件包括：`translate.c`（中间代码生成主逻辑）、`translate.h`（翻译接口）、`main.c`（集成翻译入口及输出文件控制）、`semantic.c/h`（导出语义上下文）、`syntax.y`（修正 RELOP 标记）。

## 二、总体设计与数据结构

- 变量命名映射**：维护源名称 (`x`) 到 IR 名称 (`v1`) 的映射表 `var_names[]`。每进入一个函数时清空映射，但保持 `var_counter` 全局递增，确保各函数的参数和局部变量获得全局唯一的 `vN` 编号。
- 临时变量**：使用全局递增的 `temp_counter` 生成 `t1, t2, ...` 临时变量，跨函数不重名。
- 标号管理**：使用全局递增的 `label_counter` 生成 `label1, label2, ...` 控制流标号。
- DEC 收集**：遍历函数体中的局部声明时，将数组的 DEC 信息暂存，在函数体首部统一输出。
- 数组参数标记**：`is_array_param` 标志区分"参数传入的地址"和"DEC 分配的基地址"，前者直接使用变量名作为地址值，后者也直接使用变量名（DEC 分配后 `vN` 本身就是基地址）。
- 表达式翻译结果 (TransExp)**：包含 `place`（IR 变量名或地址）、`is_addr`（是否为地址）、`type`（类型信息），二元运算/条件判断时自动对地址做解引用。

## 三、中间代码生成核心实现

### 3.1 表达式翻译 (`trans_exp`)

| 表达式类别 | 翻译策略                                                             |
|-------|------------------------------------------------------------------|
| 标识符引用 | 查符号表获取类型；数组类型返回地址 ( <code>place = vN, is_addr = 1</code> )，标量返回值 |
| 整型常量  | <code>t := #n</code> ，返回临时变量                                     |

| 表达式类别                       | 翻译策略                                                                                                                                   |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 数组下标 $a[i]$                 | 计算 $base + i * elem\_size$ , 返回地址 ( $is\_addr = 1$ )                                                                                   |
| 算术运算 $+ - * /$              | 先解引用左右操作数地址, 再 $t := x \ op \ y$                                                                                                       |
| 比较运算<br>$LT/LE/GT/GE/EQ/NE$ | 在条件上下文中直接生成跳转指令; 在值上下文生成 0/1 结果                                                                                                        |
| 逻辑 AND/OR                   | 短路求值: $AND \rightarrow IF \ left == 0 \ GOTO \ skip$ ; $OR \rightarrow IF \ left != 0 \ GOTO \ skip$                                   |
| 逻辑 NOT                      | 翻转条件跳转的真假标签                                                                                                                            |
| 一元取负 $-exp$                 | $t := \#0 - exp$                                                                                                                       |
| 函数调用                        | $read() \rightarrow READ \ t$ ; $write(exp) \rightarrow WRITE \ exp$ ; 普通调用 $\rightarrow$ 收集 ARG (顺序反转) $\rightarrow t := CALL \ func$ |
| 赋值 $lhs = rhs$              | 根据左值 $is\_addr$ 选择 $*addr := val$ 或 $var := val$                                                                                       |

### 3.2 条件与语句翻译

- **条件翻译** ( $trans\_cond$ ) : AND/OR 使用多标号实现短路求值; NOT 翻转真假标签; 其他情况求值为布尔值后跳转。
- **if 语句** :  $IF \ cond \ GOTO \ lthen \rightarrow GOTO \ lnext$ , 有 else 时多加  $GOTO \ lend$  跳过 else 块。
- **while 语句** :  $LABEL \ lbegin \rightarrow$  条件跳转  $\rightarrow$  循环体  $\rightarrow GOTO \ lbegin \rightarrow LABEL \ lend$ 。
- **return 语句** : 对返回表达式求值, 若为地址则先解引用再 RETURN。

### 3.3 函数翻译

- 函数头:  $FUNCTION \ func\_name :$
- 参数声明:  $PARAM \ vN$  (按声明顺序)
- 函数体: DEC 指令  $\rightarrow$  初始化代码  $\rightarrow$  语句序列
- 参数编号: 每函数开始时清空名称映射,  $var\_counter$  保持全局递增, 避免不同函数同名参数冲突

### 3.4 数组与高维数组

- **一维局部数组** :  $DEC \ vN \ size\_bytes$ ,  $vN$  本身代表基地址
- **一维数组参数** : 被调方 PARAM 声明, 调用方 ARG  $vN$  (直接传基地址)
- **高维数组访问** :  $a[i][j] \rightarrow$  计算  $base + i * sub\_total * 4 + j * 4$

### 3.5 错误处理

- 结构体检测: 遍历 AST 查找  $StructSpecifier$  或  $Exp \ DOT \ ID$ , 命中则 printf 错误信息并返回, 删除已生成的 .ir 文件
- 高维数组参数检测: 遍历函数形参, 多于一维则 printf 错误信息并返回

## 四、遇到的主要困难与解决方案

### 1. 跨函数变量命名冲突

- 困难：fact 和 main 中临时变量从 t1 开始编号导致重名。
- 解决：移除每函数进入时的 `temp_counter = 0` 重置，保持全局递增。

### 2. DEC 数组的多层间接寻址

- 困难：最初对 DEC 分配的数组使用 `&vN` 取地址，导致实际运行时访问错误的内存。
- 解决：理解 DEC 语义——`vN` 本身就是基地址，统一使用 `vN` 而非 `&vN`；`ARG` 也直接传 `vN`。

### 3. 函数参数跨函数重名

- 困难：不同函数同名参数（如 `x`）映射到同一个 `vN`。
- 解决：每函数开始时清空 `var_name_count`（源  $\rightarrow$  IR 映射），保持 `var_counter` 全局递增。

### 4. RELOP 标记丢失操作符信息

- 困难：syntax.y 中将所有比较运算符统一标记为 `RELOP`，翻译时无法区分 `<` 和 `>`。
- 解决：修改语法文件，为 `LT/LE/GT/GE/EQ/NE` 分别生成独立标记。

## 五、示例构建与运行命令

```
cd Lab3.0/Code
make                                # 编译生成 parser
./parser input.cmm output.ir       # 翻译单个文件
```

## 六、总结与后续工作

本次实验完成了 C-- 语言到中间代码的完整翻译器，涵盖必做全部功能及选做 4.2（高维数组 + 一维数组参数）。实现过程中最核心的工作是：设计语法制导翻译的各 `trans_*` 函数、正确理解 DEC 分配的地址语义、解决跨函数变量命名的全局唯一性问题。

后续可优化方向：（1）实现教材参考资料中的部分优化（如合并相邻标号、常量折叠）；（2）支持选做 4.1 的结构体类型翻译；（3）增加更多边界测试用例。